EXPERIMENT-31

31. C Program for Subkey Generation in CMAC

AIM

To write a C program to generate two subkeys (K1 and K2) in Cipher-based Message Authentication Code (CMAC) using a left shift operation and conditional XOR with a constant based on the block size.

PROCEDURE

1. Start the program.
2. Read the block size (64 bits or 128 bits) from the user.
3. Initialize the input block with all zero bits.
4. Apply the block cipher to the zero block to obtain the value L.
5. Generate the first subkey K1 by performing a one-bit left shift on L.
6. If the most significant bit (MSB) of L is 1, XOR the shifted result with the appropriate constant.
7. Generate the second subkey K2 by performing the same left shift and conditional XOR operation on K1.
8. Display the generated subkeys K1 and K2.
9. Stop the program.

(a) Constants Required

The constants used in CMAC subkey generation are:

Block size	Constant (Rb)	Hexadecimal
64 bits	11011	0x1B
128 bits	10000111	0x87

- For 64-bit blocks, the reduction constant is `0x1B`.
- For 128-bit blocks, the reduction constant is `0x87`.

(b) Explanation of Left Shift and XOR

Left Shift:

- Shifts every bit one position to the left.
- The most significant bit is removed.
- A zero is inserted at the rightmost position.
- This operation helps perform multiplication by \(x\) in binary polynomial arithmetic.

Conditional XOR:

- If the original MSB is 1, XOR the shifted result with the reduction constant.
- If the original MSB is 0, no XOR is required.
- This maintains the result within the required finite-field representation.

C PROGRAM

The following program demonstrates CMAC subkey generation using a sample block-cipher output L. For demonstration, it uses a user-provided hexadecimal value instead of implementing AES or DES.

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

uint64_t generate_subkey64(uint64_t input, uint64_t Rb)
{
    uint64_t msb = input >> 63;
    uint64_t result = input << 1;

    if (msb)
        result ^= Rb;

    return result;
}
```

```
void generate_subkeys(uint64_t L)
{
    uint64_t K1, K2;

    K1 = generate_subkey64(L, 0x1B);
    K2 = generate_subkey64(K1, 0x1B);

    printf("\nK1 = %016" PRIX64, K1);
    printf("\nK2 = %016" PRIX64 "\n", K2);
}

int main()
{
    uint64_t L;

    printf("CMAC Subkey Generation (64-bit)\n");
    printf("Enter 16-digit hexadecimal L: ");
    scanf("%" SCNx64, &L);

    generate_subkeys(L);

    return 0;
}
```

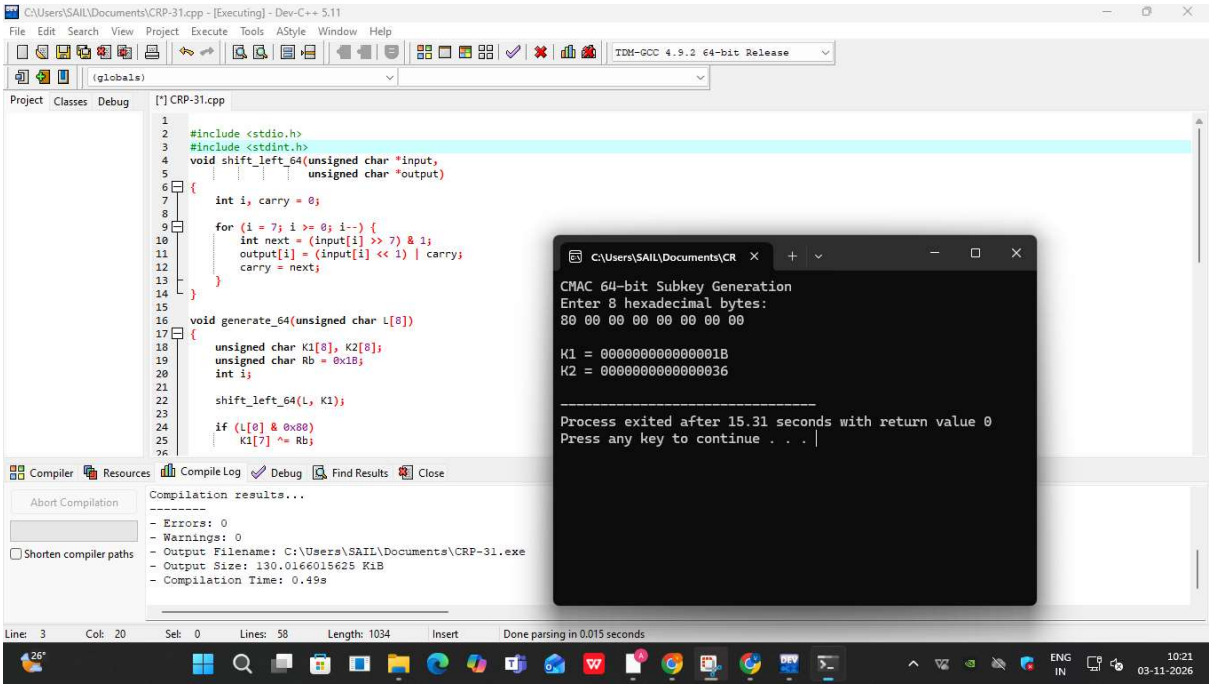
Note: This program demonstrates the subkey-generation step only. In actual CMAC, L is obtained by encrypting an all-zero block using the selected block cipher and secret key.

SAMPLE INPUT

```
CMAC Subkey Generation (64-bit)
Enter 16-digit hexadecimal L: 8000000000000000
```

SAMPLE OUTPUT

```
K1 = 000000000000001B
K2 = 0000000000000036
```



RESULT

Thus, the C program for CMAC subkey generation was successfully implemented. The two subkeys K1 and K2 were generated using left shift and conditional XOR operations. The reduction constants are `0x1B` for 64-bit blocks and `0x87` for 128-bit blocks.